

온톨로지 공부 정리노트

정다운

정리일: 2026-09-22

이 노트는 온톨로지 강의를 들으며 정리한 내용에 추가 조사한 내용을 덧붙이고, 이를 현재 진행 중인 KnowWow 아이디어에 어떻게 적용할지 생각하고 개인적으로 학습한 과정을 정리한 것입니다.

온톨로지에 대해 가장 크게 바뀐 생각

처음에는 온톨로지를 “LLM이 확률적으로 답하는 문제를 없애고 정확한 답을 주는 기술” 정도로 생각했음

공부한 뒤에는 다음을 배움

- LLM의 생성 방식 자체가 deterministic하게 바뀌는 것은 아님
- 대신 도메인에서 무엇이 무엇인지, 무엇과 무엇이 어떤 관계인지 명시적으로 정의할 수 있음
- 공식 기준, 실제 관찰 사실, SI가 발견한 패턴, 사람이 확인한 경험지식을 서로 다른 종류의 지식으로 구분할 수 있음
- LLM이 임의로 의미를 만들어내는 범위를 줄이고, 검색·검증·설명의 기준을 제공할 수 있음
- 즉 온톨로지는 SI가 참고할 도메인의 공통 의미 체계이자 가드레일에 가까움

KnowWow에 적용하면

업무 로그

↓

Observed Fact

↓

반복 패턴 탐색

↓

설명되지 않는 행동 차이 발견

↓

필요한 경우에만 Micro-interview

↓

사람이 판단 이유 설명

↓

Personal Knowledge

↓

여러 사람·사례의 공통점/차이 비교

↓

Organization Knowledge

여기서 온톨로지는 암묵지를 자동 생성하는 도구가 아님

온톨로지는 위 과정에서 나온 지식을 다음처럼 일관된 의미로 저장하기 위한 틀

CommentCase

Issue

Context

Action

Outcome

Employee
Project
Drawing
PersonalKnowledge
InferredPattern
OfficialRule

1. 별도 학습 내용을 정리하게 된 이유

LangChain 실습 프로젝트 주제를 고민하면서 조선소 상세설계 업무 경험을 다시 떠올리며

처음에는 다음과 같은 아이디어를 생각했음

1. Building Specification과 Standard Practice를 비교하는 RAG
2. 설계 변경 영향도를 분석하는 Agent
3. 과거 Comment를 검색해서 유사 처리 사례를 알려주는 Agent

그런데 계속 생각해보니 내가 실제 업무에서 느꼈던 어려움은 **문서를 못 찾는 문제**만은 아니었음

같은 Comment처럼 보여도 담당자마다 처리 방식이 달랐고, 신입 때는 여러 사람에게 물어보며 “이 사람은 왜 이렇게 처리하지?”를 익힌 뒤 나만의 방식을 만들어 갔던 기억에 착안하여 다음 질문을 떠올림

업무 시스템에 이미 쌓이고 있는 Comment 처리 이력을 이용해, 사람마다 머릿속에 있는 판단 기준을 조직 지식으로 만들 수 있을까?

여기서 다시 생긴 문제가 있음.

- Comment에는 무엇을 했는지는 남아 있음
- 하지만 왜 그렇게 했는지가 항상 남아 있는 것은 아님
- 같은 조건 상황처럼 보이는데 서로 다른 Action이 존재할 수 있음
- LLM이 그 이유를 추측하게 두면 가짜 암묵지가 만들어질 수 있음

이 문제를 정리하다 보니 **지식의 종류, Context, 출처, 관계를 어떻게 표현할 것인가**가 중요해졌고, 그 과정에서 온톨로지를 따로 공부하고 정리하고자 함

2. 온톨로지

[기업이 꼭 알아야 할 '온톨로지'의 모든 것 \(김학래 중앙대 교수\) \(티타임즈TV\)](#)

2.1 온톨로지란?

“AI를 위한 데이터”, “인간이 알고 있는 개념과 지식을 컴퓨터가 이해할 수 있도록 표현하는 방법”

특정 도메인에서 어떤 개념이 존재하고, 각 개념이 어떤 속성을 가지며, 서로 어떤 관계를 갖는지 명시적으로 정의한 지식 모델

예시:

도시 = Class

서울 = 도시의 Instance

대한민국 = 국가의 Instance

서울 --rdf:type--> 도시

서울 --locatedIn--> 대한민국

중요한 점은 단순히 서울, 대한민국이라는 문자열을 저장하는 것이 아니라 “서울은 도시이고 대한민국에 위치한 다”는 의미를 표현한다는 점

2.2 RDB와 온톨로지의 차이

RDB는 값 중심, 온톨로지는 개념과 관계 중심

조금 더 정확하게 생각하면 RDB에도 스키마·관계·제약이 있으므로 “RDB에는 의미가 없음”이라고 단순화할 수는 없음

다만 온톨로지의 관심사는 다음에 더 가까움

이 데이터가 어떤 개념인가?

이 개념과 다른 개념은 어떤 관계인가?

이 관계가 도메인에서 어떤 의미인가?

KnowFlow 예시:

CommentCase --hasIssue--> InstallationMissing

CommentCase --handledBy--> Employee

CommentCase --hasAction--> DrawingRevision

CommentCase --hasOutcome--> Accepted

DB가 값을 저장하는 방식이라면, 온톨로지는 그 값들이 업무 세계에서 어떤 의미로 연결되는지 정의하는 총

2.3 String에서 Thing으로

String → Thing

- String 중심: 같은 단어나 비슷한 문장을 찾음
- Thing 중심: 그 단어가 가리키는 실제 개념과 관계를 찾음

Google Knowledge Graph도 'things, not strings'라는 표현으로 개체와 관계 중심 검색을 설명KnowFlow에 대입하면:

"생산으로 넘김"

"생산부 이관"

"생산팀 전달"

문장은 다르지만 같은 업무 의미라면 TRANSFER_TO_PRODUCTION이라는 하나의 Action 개념에 연결 가능

반대로 다음은 비슷한 단어가 들어가도 다른 의미일 수 있음

설치 누락

설계 누락

자재 누락

따라서 “문장이 비슷하다”와 “같은 개념이다” 구분 필요

2.4 클래스, 인스턴스, 관계, 속성

온톨로지의 기본 구성

- **Class:** 공통 성격을 가진 개념. 예) CommentCase, Employee, Action

- **Instance:** 특정 Class에 속하는 실제 개체. 예) CASE-008, EMP-001
- **Relation:** 개체·개념 사이의 연결. 예) handledBy, hasAction
- **Attribute:** 개체가 가지는 값. 예) createdAt, projectId

예:

```
CASE-008 --rdf:type--> CommentCase
CASE-008 --handledBy--> EMP-001
CASE-008 --hasAction--> DRAWING_REVISION
```

2.5 LLM과 온톨로지의 관계

LLM과 온톨로지는 서로 대체 관계가 아니라 보완 관계

LLM이 잘하는 것

- 자유로운 자연어 이해
- 비슷한 표현 정리
- 긴 문서 요약
- 질문 생성
- 비정형 답변 구조화

온톨로지가 잘하는 것

- 개념의 의미를 명시함
- 서로 다른 지식 종류를 구분함
- 어떤 관계가 허용되는지 정의함
- 출처와 관계를 일관되게 연결함
- 도메인 전체의 공통 언어를 제공함

즉,

LLM = 자연어를 잘 다루는 확률적 모델

Ontology = 도메인의 의미를 명시하는 구조

2.6 온톨로지는 LLM을 deterministic하게 만드는 기술은 아님

온톨로지를 붙여도 LLM 자체는 여전히 확률적으로 문장을 생성

대신 다음처럼 구조 분리

자연어 입력



LLM이 개념 매핑



Ontology / KG / Rule / Code에서 사실 조회



확인된 결과



LLM이 자연어로 설명

따라서 온톨로지는 LLM의 확률적 성질을 없애는 것이 아니라 LLM이 마음대로 결정해야 하는 영역을 줄이는 역할

3. 온톨로지, 지식 그래프, RDF가 헛갈렸던 부분

3.1 온톨로지 vs 지식 그래프

Ontology

= 어떤 개념과 관계가 존재할 수 있는지 정의

Knowledge Graph

= 그 구조를 이용해 실제 개체와 사실을 연결

예:

Ontology

CommentCase라는 개념이 있음

Employee라는 개념이 있음

CommentCase --handledBy--> Employee 관계를 정의함

Knowledge Graph

CASE-008 --handledBy--> EMP-001

CASE-008 --hasAction--> DRAWING_REVISION

단, 실제 시스템에서는 온톨로지와 인스턴스 데이터가 완전히 따로 떨어져 있어야 한다는 뜻은 아니며 역할을 이해하기 위한 구분

3.2 RDF

RDF는 정보를 주어-서술어-목적어의 트리플로 표현하는 W3C 표준 데이터 모델

CASE-008 | handledBy | EMP-001

CASE-008 | hasAction | DRAWING_REVISION

온톨로지뿐 아니라 실제 데이터를 표현하는 데도 사용할 수 있음

3.3 OWL

OWL은 클래스와 속성의 의미, 관계, 공리 등을 더 풍부하게 표현하는 언어임

예를 들면:

InstallationMissing is-a Issue

DrawingRevision is-a Action

같은 분류와 논리적 관계를 더 명시적으로 표현

3.4 SHACL

SHACL은 RDF 데이터가 원하는 형태와 제약을 지키는지 검사하는 데 활용할 수 있음

KnowWow에 적용한다면 다음 같은 검증을 생각할 수 있음

HumanConfirmedKnowledge라면

- source_case가 있어야 함
- confirmed_by가 있어야 함
- raw_evidence가 있어야 함

이런 검증은 LLM에게 “잘 지켜줘”라고 부탁하는 것보다 데이터 구조에서 강제하는 편이 더 안전

3.5 PROV-O

PROV-O는 지식이나 데이터가 어디에서 유래했는지 표현하는 데 참고할 수 있음

KnowWow에서는 특히 중요함.

이 Knowledge는 누구의 설명인가?

어떤 Comment에서 출발했는가?

누가 확인했는가?

언제 만들어졌는가?

암묵지 시스템에서 출처가 없는 지식은 위험

4. 특정 도메인에서 온톨로지가 중요한 이유

4.1 같은 단어보다 같은 의미가 중요함

제조 분야에서는 시스템마다 같은 것을 다른 용어로 표현하거나, 반대로 같은 단어를 다른 의미로 사용하는 경우가 많음

또한 현장에서 사용하는 용어가 별도로 존재함

Industrial Ontologies Foundry(IOF) 관련 NIST 연구도 제조 정보의 상호운용성을 위해 공통 의미 기반이 필요하다는 문제를 다룸

예를 들어 Material Available도 실제 업무에서는 여러 뜻일 수 있음

창고에 재고가 있음

호선에 할당됨

현장에 반출됨

설치할 수 있는 상태임

이걸 모두 AVAILABLE 하나로 합쳐버리면 데이터는 깔끔해 보여도 실제 판단에는 쓸 수 없을 수 있음

→ 온톨로지 구축에서 중요한 것은 데이터를 많이 연결하는 것이 아니라, 현업에서 구분해야 하는 차이를 정확히 남기는 것

4.2 계획과 실제 상태를 구분해야 함

제조에서는 디지털 자료와 물리적인 실제 상태가 다를 수 있음

NIST 연구에서도 디지털 산출물과 물리적 대상의 관계를 의미적으로 표현하는 문제가 다뤄짐

조선 업무 예시로 생각하면:

도면에 표시되어 있음

≠

실제 현장에 설치되어 있음

도면이 최신임

≠

현재 위치에 실제 설치 가능함

이 차이를 같은 상태로 모델링하면 Agent가 잘못된 판단을 할 수 있음

4.3 제조 문서는 원래 복잡함

제조 문서는 텍스트만 있는 것이 아니라 표, 이미지, 기술 도면, 수식 등이 섞여 있음

OmEGa 연구는 제조 문서에서 작업 중심 온톨로지를 이용해 개체·관계를 추출하고 지식 그래프를 구성하는 접근을 제안함

이번 과제에서는 이 부분까지 구현하지 않고, DX가 끝나서 구조화된 데이터가 있다고 가정하기로 함

4.4 조선업도 RAG·Agent·지식모델 연구가 진행 중임

조선 엔지니어링 문서에 RAG와 Multi-Agent를 적용하는 연구가 2026년에 발표됨

- 긴 Engineering Document를 단순 Vector Search 하나로 처리하기 어려움
- 검색, 정리, 추가 정보 확인 등의 역할을 나눌 수 있음
- 다만 복잡한 계산과 최종 기술 판단은 여전히 검증 필요

5. 온톨로지와 LLM을 같이 쓸 때 주의할 점

5.1 Grounding을 해도 원본 지식이 틀리면 소용없음

최근 연구를 보면 Knowledge Graph를 LLM이 만드는 추세지만

LLM이 만든 Knowledge Graph

↓

그 그래프를 다시 LLM의 Grounding에 사용

했는데 처음 그래프가 틀렸다면 **환각을 환각으로 검증하는 구조**가 될 수 있음

따라서 KnowFlow에서는 다음 원칙이 필요

- 원문 업무 이력을 보존
- LLM이 추출한 값에는 source를 연결
- AI가 추론한 패턴과 사람이 확인한 지식을 구분
- 공식 기준은 출처 문서와 버전을 따로 관리

5.2 온톨로지가 있어도 공식 Rule과 경험 패턴은 다름

이것도 프로젝트 설계에서 중요했음

과거 20건 중 15건이 생산 이관이라는 결과는 다음을 의미함

“이 조건에서 생산 이관이 많이 관찰됨”

하지만 다음을 의미하지는 않음.

“이 조건에서는 반드시 생산으로 이관해야 함”

따라서 경험 데이터가 다수 발생했다고 해서 공식 Rule로 자동 승격하면 안 됨

6. 암묵지

6.1 Inferred Pattern 자체가 암묵지는 아님

처음에는 반복 패턴을 찾으면 그게 암묵지라고 생각했음

지금은 다음처럼 구분함.

Observed Fact

= 실제 업무 기록에 남아 있는 사실

Inferred Pattern

= 여러 사례에서 관찰되는 반복 경향

Tacit Knowledge

= 기록에 직접 적혀 있지 않은 판단 기준, Why, 예외, 중요 Context

예:

CASE A

Issue = InstallationMissing

Material = Available

Drawing = Valid

Action = ProductionTransfer

CASE B

Issue = InstallationMissing

Material = Available

Drawing = Valid

Action = DrawingRevision

현재 보이는 데이터만 보면 같은 조건인데 Action이 다름

이때 “왜 다르지?”라는 **Knowledge Gap**이 생김

담당자에게 물어봤더니:

“CASE B는 실제 설치 위치를 다른 장비가 점유하고 있어서 그 위치에 설치할 수 없었음.”

이라고 설명한다면 숨어 있던 Context가 드러남.

InstallationFeasibility

그리고 실제 판단 기준도 드러남

단순 설치 누락이라도 실제 설치 가능 여부를 확인해야 함

이것이 암묵지가 명시지로 바뀌는 과정에 더 가깝다고 생각함

6.2 암묵지로 볼 수 있는 것

KnowWow에서 다루고 싶은 암묵지는 주로 언어로 외재화 가능한 Engineering Judgment

예:

- 어떤 Context를 먼저 확인하는가

- 어떤 조건을 중요하게 보는가
- 왜 평소 Pattern과 다른 Action을 선택했는가
- 어떤 상황에서는 일반적인 방법을 적용하지 않는가
- 업무 중 자주 놓치는 것은 무엇인가
- 과거 비슷한 호선에서 어떤 판단을 했는가

반대로 손의 감각, 숙련 조작 등 언어로 설명하기 어려운 신체적 암묵지는 이번 범위에서 제외함

7. 암묵지는 누가 뽑는가? → LLM + 사람

7.1 역할을 나눠야 함

LLM 혼자 암묵지를 “복원”하게 하면 안 됨

- 반복 패턴 집계 → 일반 코드 / 통계가 담당
- 설명되지 않는 Action 차이 탐지 → 코드 + 규칙이 담당
- 담당자에게 질문 생성 → LLM이 담당
- 실제 판단 이유 제공 → 사람이 담당
- 자유서술 답변 구조화 → LLM이 담당
- 구조화 결과 확인 → 사람이 담당
- 개인 지식 저장 → 시스템이 담당
- 여러 개인 지식 비교 → 코드 / 분석 로직이 담당
- 새로운 개념 후보 제안 → LLM이 보조 가능
- 온톨로지 최종 개념 추가 → 도메인 전문가가 검토

LLM은 사람이 가지고 있던 암묵지를 끌어내고 구조화하는 역할

7.2 질문은 최소화해야 함

모든 Comment마다 “왜 그렇게 했나요?”라고 물으면 현업에서 실질적으로 사용하기 어려움

그래서 다음 방식으로 정리

기존 Pattern으로 설명 가능

→ 질문 안 함

기존 Stable Pattern과 다른 Action 발생

→ Micro-interview

예:

“과거 유사 사례에서는 대부분 생산부서 이관으로 처리됐는데, 이번 건은 도면 개정으로 처리했습니다. 기존 사례와 달랐던 핵심 조건은 무엇이었나요?”

이 방식이 내가 생각하는 **Selective Tacit Knowledge Capture**의 핵심

8. 개인 지식을 따로 저장해야 하는 이유

실제 업무에서는 사람마다 일하는 방식이 다름

예:

김OO

자재 있음 + 도면 정상

→ 바로 생산 이관

박OO

자재 있음 + 도면 정상

→ 현장 확인 후 생산 이관

이OO

자재 있음 + 도면 정상

→ 설계에서 한 번 더 확인 후 이관

이 차이를 처음부터 하나의 조직 Rule로 합치면 중요한 정보가 사라짐

그래서 개인 지식은 따로 보존할 필요가 있음.

Employee

└ OWNS_KNOWLEDGE

└ PersonalKnowledge

- 한 사람이 반복해서 사용하는 판단인지 알 수 있음
- 여러 사람에게서 공통적으로 나타나는지 비교 가능
- 같은 상황에서 서로 다른 행동이 나왔을 때 새로운 Context 후보를 찾을 수 있음
- “누가 그렇게 생각했는가?”라는 출처를 유지

8.1 개인별 DB를 따로 만드는 것은 아님

직원마다 물리적인 Graph DB를 하나씩 만드는 것보다 하나의 Enterprise Knowledge Graph 안에서 논리적으로 분리하는 구조가 자연스럽다고 생각함

Personal Knowledge Layer

Team Knowledge Layer

Organization Knowledge Layer

Official Knowledge Layer

각 지식에는 owner, source_case, sharing_scope 등을 연결

9. Text에서 반복 패턴으로 바꾸는 과정

개인 지식은 처음에는 자연어 Text로 들어올 가능성이 높음

예:

“자재는 이미 현장에 나와 있었고 최신 출도 도면에도 정상 반영돼 있어서 설계 문제는 아니라고 봤습니다. 그래서 생산으로 넘겼습니다.”

이걸 바로 조직 규칙으로 만들면 안 됨.

9.1 원문 보존

Raw Answer

"자재는 이미 현장에 ..."

원문은 반드시 남겨야 함

이유:

- LLM이 잘못 구조화했을 때 다시 확인 가능
- 사람이 말한 범위를 넘어 해석했는지 확인 가능
- 나중에 모델링 방식이 바뀌어도 다시 처리 가능

9.2 LLM이 구조화

```
{
  "situation": "INSTALLATION_MISSING",
  "contexts": {
    "material_status": "AVAILABLE",
    "drawing_status": "VALID"
  },
  "decision": "NOT_DESIGN_ISSUE",
  "action": "TRANSFER_TO_PRODUCTION",
  "rationale": "설계와 자재 조건이 이미 충족되어 있었음"
}
```

여기서도 사용자가 말하지 않은 값은 null로 두는 것이 원칙

9.3 Ontology 기준으로 정규화

사람마다 표현이 다를 수 있음

"도면은 이상 없었음"

"최종 출도도 정상"

"Drawing에는 반영돼 있었음"

업무적으로 같은 의미라면:

DrawingStatus = VALID

로 정규화

단, 현업에서 실제로 다른 의미라면 하나로 합치면 안 됨

9.4 반복 패턴 집계

정규화된 데이터가 쌓이면 일반 코드로 집계 = 관찰된 통계

조건:

InstallationMissing

Material = Available

Drawing = Valid

Observed Actions:

ProductionTransfer 18

SiteCheckThenTransfer 5

DrawingRevision 3

9.5 설명되지 않는 차이를 다시 질문

만약 동일한 Context에서 특정 직원만 다른 Action을 반복한다면:

“같은 조건으로 기록된 사례와 다른 방식으로 처리한 경우가 반복됩니다. 이 두 상황을 구분하는 추가 조건이 있었나요?”

처럼 질문할 수 있음.

이 과정이 반복되면서 새로운 Context를 발견할 수 있음

10. 온톨로지는 어떻게 성장하는가

새로운 경험 하나가 생겼다고 온톨로지에 새 Class를 바로 추가하는 것은 아님

10.1 대부분은 데이터 추가

예를 들어 InstallationFeasibility라는 개념이 이미 온톨로지에 있다면:

CASE-008

hasInstallationFeasibility = IMPOSSIBLE

이라는 값만 추가하면 됨

이건 Knowledge Graph의 데이터가 늘어난 것이지 온톨로지가 바뀐 것은 아님

10.2 기존 구조에 없는 개념이면 Candidate로 저장

직원이:

“작업 구역에 다른 팀이 작업 중이어서 실제 설치가 가능한 상태인지 먼저 봤음.”

이라고 설명했는데 현재 모델에 관련 개념이 없다면:

ContextCandidate

= WorkAreaReadiness

정도로 저장

10.3 반복·중요성 검토 후 온톨로지 후보가 됨

다른 직원·다른 Case에서도 같은 의미가 반복되면:

WorkAreaReadiness

- 4명의 직원에게서 언급
- 17개의 관련 Case 존재
- 여러 프로젝트에서 반복

같은 근거가 생김

그때 도메인 전문가가 다음을 검토함.

- 기존 개념과 중복되는가
- 정말 독립적인 Context인가

- 어떤 값들을 가져야 하는가
- 어떤 Case에 적용되는가
- 다른 관계와 어떻게 연결되는가

승인되면 온톨로지에 추가함.

Context

- └ MaterialStatus
- └ DrawingStatus
- └ InstallationFeasibility
- └ WorkAreaReadiness ← 추가

즉:

LLM = Ontology Change Proposal 보조

사람 = Ontology Change Approval

11. KnowWow 최종 온톨로지 방향

11.1 핵심 질문

온톨로지를 만들기 전에 “무슨 질문에 답하고 싶은가?”를 먼저 정해야 함

KnowWow에서 중요한 질문은 다음과 같음

- 이 Comment는 어떤 Issue인가
- 당시 어떤 Context가 기록되어 있었는가
- 누가 어떤 Action을 했는가
- 결과는 어땠는가
- 비슷한 조건에서 다른 사람들은 어떻게 처리했는가
- 이 판단에 대해 담당자가 직접 설명한 지식이 있는가
- 어떤 지식은 공식 Rule이고, 어떤 지식은 경험 패턴인가
- 이 지식은 어떤 Case에서 유래했는가
- 현재 상황과 과거 사례의 차이는 무엇인가

이 질문에 답하기 위한 범위만 우선 모델링

11.2 핵심 Class 후보

업무 영역

CommentCase

Issue

Context

Action

Outcome

Employee

Department

Project
Vessel
Drawing
Material
StandardPractice

지식 영역

ObservedFact
InferredPattern
PersonalKnowledge
OrganizationalKnowledge
OfficialRule
KnowledgeGap
ContextCandidate

11.3 핵심 Relation 후보

CommentCase --hasIssue--> Issue
CommentCase --hasContext--> Context
CommentCase --handledBy--> Employee
CommentCase --hasAction--> Action
CommentCase --hasOutcome--> Outcome
CommentCase --occurredIn--> Project
CommentCase --relatedTo--> Drawing

PersonalKnowledge --ownedBy--> Employee
PersonalKnowledge --derivedFrom--> CommentCase
PersonalKnowledge --mentionsContext--> Context

InferredPattern --supportedBy--> CommentCase
OrganizationalKnowledge --supportedBy--> PersonalKnowledge
OfficialRule --definedBy--> StandardPractice

11.4 지식 종류를 반드시 분리

이 부분이 최종 설계에서 가장 중요

- **ObservedFact**: 실제 업무 시스템에 기록된 사실
- **InferredPattern**: 여러 Case에서 시스템이 발견한 반복 경향
- **PersonalKnowledge**: 특정 직원이 직접 확인한 판단 근거
- **OrganizationalKnowledge**: 여러 사람·사례에서 의미가 확인된 조직 지식
- **OfficialRule**: 회사 Standard·Procedure 등 공식 기준

예:

OBSERVED
CASE-008 → DRAWING_REVISION

INFERRED
동일 Context 20건 중 15건 ProductionTransfer

PERSONAL

EMP-001: 실제 설치 불가능한 경우 DrawingRevision을 검토했음

OFFICIAL

SP-014: 회사의 공식 설치 기준

이들을 같은 수준의 “정답”으로 취급하지 않는 것이 핵심

12. 개인지식과 조직지식은 어떻게 연결되는가

개인지식을 곧바로 조직지식으로 바꾸지 않음

Personal Knowledge



여러 직원·사례 비교



공통점 / 차이점



Organization Pattern Candidate



검증



Organizational Knowledge

반복을 볼 때 단순 반복 건수만 보지 않음

- 한 사람이 몇 번 반복했는가
- 몇 명의 직원에게서 나타났는가
- 몇 개의 프로젝트에서 나타났는가
- 특정 선종에만 나타나는가
- 최근에도 계속 나타나는가
- 어떤 Outcome과 함께 나타났는가

등을 같이 볼 필요가 있음

그리고 반복이 많다고 곧 정답이라는 뜻은 아님

13. 우리 프로젝트에 온톨로지가 왜 맞는가

13.1 사람마다 다른 표현을 정리할 수 있음

"생산으로 넘김"

"생산부 이관"

"생산팀 처리"

→ 같은 의미라면 하나의 Action 개념으로 정규화 가능

13.2 사람마다 다른 업무 방식을 없애지 않고 비교할 수 있음

EMP-001 → A 방식

EMP-002 → B 방식

차이를 그대로 보존하면서 공통 Context와 다른 Context를 분석할 수 있음

13.3 LLM이 추론한 것과 사람이 확인한 것을 구분할 수 있음

InferredPattern ≠ HumanConfirmedKnowledge

이 구분 자체가 LLM 환각 방지에 중요

13.4 Context가 빠졌다는 사실을 알 수 있음

Ontology에 현재 알려진 Context가 정의되어 있다면:

MaterialStatus = ?

DrawingStatus = VALID

InstallationFeasibility = ?

처럼 무엇을 모르는지 표현할 수 있음.

그러면 Agent 질문도 더 구체적으로 만들 수 있음

“당시 필요한 자재는 이미 현장에 확보되어 있었나요?”

13.5 지식의 출처를 추적할 수 있음

이 판단은 누구의 설명인가

어떤 Case에서 나온 것인가

누가 확인했는가

어떤 공식 기준과 관련되는가

14. 이번 LangChain 과제에서의 적용

이번 과제에서는 온톨로지 전체를 구현하지 않으며 구조는 그대로 두고 저장 형식만 JSON으로 단순화

Ontology / KG

→ JSON + Controlled Vocabulary + ID Reference

예:

```
{
  "knowledge_id": "PK-001",
  "employee_id": "EMP-001",
  "source_case_ids": ["CASE-008"],
  "raw_answer_text": "지정 위치를 다른 장비가 점유해 위치 변경이 필요했음.",
  "structured_knowledge": {
    "new_context": {
      "name": "installation_feasibility",
      "value": "IMPOSSIBLE"
    },
    "action": "DRAWING_REVISION",
    "rationale": "지정 위치에 설치할 수 없어 위치 변경이 필요했음"
```

```

},
"verified_by_user": true
}

```

이번 MVP에서 구현할 것

- 업무 Comment 더미 데이터
- 반복 Action 패턴 계산
- Pattern Gap 탐지
- Gap일 때만 LLM Micro-question
- 사람이 답한 이유를 Structured Output으로 저장
- 개인 지식 / 조직 패턴 / 과거 Case RAG
- Evidence ID 포함 답변

후속 온톨로지 프로젝트에서 할 것

- RDF / OWL
- Knowledge Graph
- SHACL 검증
- 개인 / 조직 / 공식 지식 Layer
- Context Candidate → Ontology Evolution
- GraphRAG / Hybrid RAG

15. 온톨로지 구축 흐름 정리

1. 도메인의 목적과 질문 정의
- ↓
2. 핵심 개념 정의
- ↓
3. 속성과 관계 정의
- ↓
4. 기존 업무 데이터를 개념에 매핑
- ↓
5. 실제 Case / 지식을 연결
- ↓
6. 사용하면서 설명되지 않는 공백 발견
- ↓
7. 사람에게 필요한 정보만 확인
- ↓
8. 새로운 Knowledge 추가
- ↓
9. 기존 모델로 표현이 안 되면 Concept Candidate 생성
- ↓
10. 전문가 검토 후 Ontology 확장

중요한 점은 온톨로지가 처음부터 완벽하게 만들어지는 것이 아니라, 실제 업무에서 필요한 질문을 해결하면서 반복적으로 발전할 수 있다는 점

16. 최종 정리

16.1 온톨로지는 정답표가 아님

처음에는 온톨로지가 기업의 정확한 Rule을 모두 넣어놓은 확정된 기준표라고 생각했지만

실제로는 무엇이 Fact이고, 무엇이 Pattern이고, 무엇이 개인의 설명이고, 무엇이 공식 기준인지 구분하게 해주는 의미 구조라는 점을 배웠음

16.2 암묵지를 직접 SI가 알아내는 것은 아님

업무 로그만 보고 LLM이 판단 이유를 만들어내면 위험함

따라서:

데이터에서 공백 발견

→ LLM이 질문

→ 사람이 설명

→ LLM이 구조화

→ 사람이 확인

이라는 Human-in-the-loop 구조가 필요

16.3 개인차도 지식임

같은 업무를 사람마다 다르게 처리하는 것이 단순히 “누가 틀림”을 의미하지 않을 수 있음

오히려 그 차이가 기록되지 않은 Context를 발견할 단서가 될 수 있음

그래서 개인 지식을 지우지 않고 남기는 구조가 중요하다고 생각

16.4 KnowWow에서 온톨로지의 역할

KnowWow에서 온톨로지는 사람의 판단을 대신하는 규칙집이 아니라, 업무에서 관찰한 사실·반복 패턴·개인의 판단 근거·공식 기준을 서로 섞지 않고 연결하기 위한 공통 의미 구조

그리고 LLM은 그 위에서:

- 자연어를 이해하고
- 필요한 순간에 질문하고
- 답변을 구조화하고
- 근거가 있는 지식을 찾아 설명하는 역할을 맡게 됨.

이번 LangChain 실습에서는 이 구조를 JSON으로 축소 구현하고, 이후 별도 프로젝트에서 실제 온톨로지와 Knowledge Graph까지 확장해보고자 함
